PyTorch vs Phinet, and 64 Practice Topics

Phiano Project · companion to Papers 1–3 · September 2026

> **The paradigm.** In Phient's world, a word *w* is nothing but what it relates to, under what conditions *c*. We call it the Phical paradigm, **X ⇒ Y**; it drives and composes Phient's Phasor Language Models — Phinet, Phiano, PhiTon. Phical stores no point: a word is 32–64 relations, each a phasor on a closed circle with a count of how often it has been played out, and meaning is the side effect of pairs of words moving under a condition. Everything below is one way of writing that down (Paper 1 §1 states it formally).

Every Phinet example below compiles against `gemphi/phinet` as it is today (`use phinet::*`); every Phical example calls functions that exist in `phiano/src/phical`, `phiton`, `phinum`, or `persona`, except where the text says *proposed*. Numbers quoted are the measured ones from Papers 1–2.

Part A — Compare and contrast

PyTorch is a language for differentiable programs; phinet is one sentence in it, spoken in Rust

aspect	PyTorch	phinet
unit of data	real/complex Tensor, any rank	Tensor [batch, width, space] of C^{64}
autograd	dynamic tape, ~2,000 ops	dynamic Tape, 17 ops, all finite-difference checked
complex gradients	Wirtinger, conj convention	g = ∂L/∂Re + i·∂L/∂Im (documented, one convention)
FFT	torch.fft, cuFFT/MKL	own unitary radix-2, power-of-two spaces, differentiable
the operator	you assemble nn.Modules	Fno is the primitive; layers, modes, width are config
ranking losses	write your own / torch.nn	hinge, cosine_table, mixture_nll built in
training the table	nn.Embedding + optimizer	Joint: table as tape parameter with proximal tether
composition	not a primitive	hrr::bind/unbind/compose on the circle
hardware	CUDA, MPS, XLA, CPU	CPU only, single core today (parallel cosine_table is next)
size	~ 10^6 lines, Python + C++	~ 2,000 lines of Rust, two dependencies
persistence	pickle / safetensors	to_bytes/from_bytes (bincode), .fno files
what it is for	any differentiable program	functions on a closed circle, ranked against a table

A.1 What they are

PyTorch is a general-purpose differentiable programming system: a tensor library with broadcasting over arbitrary ranks and dtypes, a dynamic autograd engine over ~2,000 operators, backends for CUDA/ROCm/MPS/XLA/CPU, a

module system (`nn.Module`), optimisers, data loaders, distributed training, JIT compilation, quantisation, and a ten-year ecosystem. Its unit of thought is "a differentiable program over tensors".

Phinet is one architecture with its training loops: a complex `Tensor [batch, width, space]`, a dynamic `Tape` with 17 operators, a unitary FFT, a Fourier neural operator, three objectives (hinge, mixture NLL, joint with tether), and holographic bind/unbind — about 2,000 lines of Rust with two dependencies. Its unit of thought is "a function on a closed circle, ranked against a table of functions".

A.2 Where they agree

- **Dynamic tape.** Both record operations as they execute and replay them backward; there is no static graph in either.
- **Complex gradients.** PyTorch uses the Wirtinger/conjugate convention (`grad = ∂L/∂z̄`, scaled); phinet uses `g = ∂L/∂Re + i·∂L/∂Im`. They differ by a factor of 2 and a conjugation and agree on every descent direction. Phinet documents its single convention in one place and finite-difference-checks every op against it.
- **FNO.** phinet's operator is the Li et al. FNO: lift, L Fourier layers (spectral diagonal on low modes + pointwise linear, GELU), project. A PyTorch FNO from `neuraloperator` has the same forward pass; the two agree on the parameter count at equal (width, layers, modes).
- **Adam.** Same update, same defaults.

A.3 Where they differ

	PyTorch	Phinet
domain	any tensor shape	`[batch, width, space]`, space a power of two
FFT	`torch.fft.*`, any length, any axis	own radix-2, one axis, unitary
what "a model" is	anything you compose	`Fno` (+ `Joint`)
objectives	you write them, or `torch.nn.*`	`hinge`, `cosine_table` + `mixture_nll` built in and checked
training the table	`nn.Embedding` + your optimiser	`Joint` with proximal tether, separate Adam rates
composition	not a primitive	`hrr::bind/unbind/compose` on the circle
hardware	GPUs, TPUs, many cores	CPU; `cosine_table` on all cores (rayon), the rest single-threaded
deployment	Python runtime, or TorchScript/ONNX	a Rust crate inside the lexicon's process; `.fno` files
verification	community-tested ops	every op finite-difference tested in `cargo test`
speed at our sizes	slower to *start* (Python, kernel launch)	20 s/epoch hinge, 7 min/epoch mixture on 6k words
speed at large sizes	vastly faster	not competitive
size	~10^6 lines	~2,000 lines

A.4 When to use which

Use PyTorch when you want a *second* architecture, a GPU, a dataset over ten million tokens, or anyone else to reproduce you. Use phinet when you want *this* architecture inside Phiano: the table and the operator on one tape, evidence-weighted functions from the lexicon fed straight in, a persona as a tethered table copy, and no Python between the lexicon and its reader. The `neuraloperator` PyTorch FNO can be used to *check* phinet: same config, same data, same MRR within noise is the acceptance test, and it has not been run yet.

A.5 What phinet should copy from PyTorch next

1. ~~A parallel `cosine_table` (Rayon)~~ — done (86 → 51 ms per step on two cores). Next: the FFT and `linear` over the batch.
2. `Tensor` of rank > 3 is not needed; batched *tables* are (persona copies).
3. A `Module` trait so `SphFno`, `Mobius`, `Fno` compose the way `nn.Sequential` does.
4. `safetensors`-style export so a phinet operator can be loaded into PyTorch for the acceptance test above.

Part B — 32 Phinet practice topics

```
        use phinet::{Fno, FnoConfig, Adam, Example, MixtureExample, Joint,
IndexExample,
                     Function, Tape, Tensor, c64, rank, cosine, unit_random,
unit_phase, normalise, hrr};
```

B1 · A function on the circle. `let f: Function = unit_random(64, 1);` — 64 complex samples, unit energy. Everything in phinet is a `Vec<c64>` of a power-of-two length.

B2 · Cosine between functions. `cosine(&f, &g)` is `Re(f,g)/(‖f‖‖g‖)`. It is the similarity every objective uses; in Phiano it is `similarity_with` once evidence weights are folded into the samples (`phiano::phinet::space::encode`).

B3 · Configure an operator. `FnoConfig { space: 64, width: 4, layers: 2, modes: 32, margin: 0.5, lr: 1e-2 }` — 8,360 parameters. `FnoConfig::for_space(64)` gives the defaults used in Paper 2.

B4 · Build and count. `let mut fno = Fno::new(cfg, seed);` `fno.parameter_count()`. Seeds make every run reproducible; the papers use one seed and say so.

B5 · Apply without training. `fno.apply(&inputs)` returns one output function per input. A fresh operator is a random spectral filter; `rank` it anyway to get a chance baseline before training.

B6 · The identity baseline. Skip the operator: `rank(space.table(), &inputs, &examples, &pool)`. On WikiText-2 this is 6.7× chance one-sided and **15.5×** with `Sides::Directed` (source context, target table); it is the number every operator has to beat.

B7 · Hinge training. `fno.train_epoch(&table, &examples, &pool, 20, &mut Adam::new(cfg.lr), epoch)` — one uniform decoy per example, margin 0.5. 20 s/epoch on 6k words.

B8 · Early stopping. Track `rank(...)` on a held-out split every epoch and keep the best `to_bytes()`. The operator overfits between epochs (149 → 161 without it).

B9 · A custom decoy sampler. `fno.train_epoch_with(&table, &examples, &|target, seed| my_pick(target, seed), 20, &mut adam, epoch)`. `FrequencySampler::new(items, &weights, 0.75)` is the built-in frequency-proportional one — and it made things worse; keep uniform unless you measure otherwise.

B10 · Mixture-likelihood training. Build `MixtureExample { input, target, k, coef, other }` per position from the n-gram harness (`PhianoLM::mixture_terms`), then `fno.train_epoch_mixture(&table, &ex, gamma, 8.0, 20, &mut adam, epoch)`. Returns mean NLL; `exp` of it is the operator's own perplexity. Worth 15 points over hinge.

B11 · Residual training. Same call with `gamma = 0.5` or `0.2`: the operator is trained only on what the unigram misses. Expect its own perplexity to worsen and the mixture to improve by a fraction of a point.

B12 · Reading the mixture. `p = k + coef·(γ·softmax(β·cos)[w] + (1−γ)·uni[w])`. Sweep γ on validation (`PhianoLM::gamma_sweep_scored`); the best γ was 0.1 on the Rust Book. If it is 0, the operator adds nothing.

B13 · Save and load. `std::fs::write("op.fno", fno.to_bytes());` `Fno::from_bytes(&bytes)`. Bincode; the config is inside the file.

B14 · Joint training of operator and table. `let mut j = Joint::new(fno, &table, window).with_tether(0.02); j.train_epoch(&idx_examples, &pool, 20, &mut adam_op, &mut adam_table, seed)` with `adam_table` at ¼ the rate. `j.functions()` returns the moved table; `phiano::phinet::joint::write_back` folds it into the lexicon.

B15 · Why the tether. Set `tether = 0.0` and watch validation perplexity go 149 → 157: the table memorises. 0.02 gave 134.1.

B16 · Roles. `let r = hrr::role(64, seed)` — unit-magnitude random phases. Binding is phase addition: `hrr::bind(&f, &r); hrr::unbind(&hrr::bind(&f,&r), &r) == f` exactly.

B17 · Compose and recover. `let c = hrr::compose(&[bind(a,r1), bind(b,r2), bind(c,r3)]); cosine(&unbind(&c,&r1), &a)` — high but not 1; the other two terms are crosstalk. At depth 1 phase subtraction alone ranks the constituent at 19× chance among 704 words.

B18 · Train a cleanup. Examples `Example { input: unbind(c, r_j), target: j }` for every composition and constituent; hinge-train an `Fno` on them. 66× chance at level 1, 52× at 2, 45× at 3 when trained on levels 1..3.

B19 · Depth is a training decision. Train on levels 1..K and test on K+1: chance. Curriculum 1→2→3 gives 7× at level 4. Nothing in the architecture limits depth; the data does.

B20 · One operator, two tasks. Interleave LM examples and cleanup examples in one `Fno`; measure both. 2 % off on LM, 4–8 % off on recovery, same perplexity.

B21 · Mask modes. `fno.mask_modes()` zeroes spectral weights above `modes`; call it after loading an operator trained with more modes to test how much the high frequencies carry.

B22 · Reset. `fno.reset()` re-initialises parameters from the seed without rebuilding the tape — for repeated trials in one process.

B23 · The tape by hand. `let mut t = Tape::new(); let x = t.constant(Tensor::randn([1,1,64],1.0,1)); let w = t.param(Tensor::randn([1,4,1],0.1,2)); let h = t.linear(x, w); let y = t.gelu(h); let l = t.mean_real(y); t.backward(l); t.grad(w)`. This is all `Fno::forward` does, unrolled.

B24 · The FFT is unitary. `let f = t.fft(x); let b = t.ifft(f);` — `value(b) == value(x)` to 1e-15, and `‖fft(x)‖ = ‖x‖`. This is why spectral and pointwise weights share a scale.

B25 · Finite-difference check your own op. Copy the pattern in `tensor.rs` tests: perturb one real or imaginary component by ε, compare `(L(z+ε) − L(z−ε))/2ε` to the tape's gradient. Every shipped op passes at 1e-6.

B26 · Cosine against a whole table. `t.cosine_table(pred, table)` → `[batch, V, 1]` of cosines with gradients to both. It is the full-vocabulary softmax's input and the thing that needs Rayon.

B27 · Encode a Phiano word. `phiano::phinet::space::encode(&phasor)` → unit function whose sample k is `ev(k)·e^{iφ_k}` normalised. Context windows are weighted sums of encoded words (`1/distance`).

B28 · Inspect a model file. `cargo run --bin phinet -- read data/manifold.chroma` prints word count, dispersion, evidence coverage, marker table. On the v1 manifold: 216,097 words, dispersion 0.288, no evidence.

B29 · Guide generation with the operator. `Generator::new(32, 0.15).with_operator(OperatorGuide::load("data/phinet_generate.fno", &facet, Sides::Directed, 6)?)` — every candidate's score is multiplied by `0.5 + 0.5·(1+cos)/2` against the operator's prediction; `command generate` does this automatically when the file exists and the lexicon carries evidence.

B30 · Persona as a tethered table (proposed). `Joint::new(shared_fno, &table, window).with_tether(0.02)`, trained on one persona's `IndexExample`s only; keep `j.functions()` as that persona's table. Deleting it is exact unlearning.

B31 · Möbius layer. `let m = t.param(Tensor::from_vec([1,1,4], vec![one, zero, zero, one])); let w = t.mobius(x, m);` — `w = (az+b)/(cz+d)` per sample, gradients to `x` and to all four coefficients (finite-difference checked). Initialised to identity it is a no-op; a rotation is bind. Also: `fno.scale_modes(&phinet::zeta_weights(33, 1.5))` after each step applies the zeta smoothness prior.

B32 · Acceptance test against PyTorch (proposed). Export the Rust Book examples as `.npy`, train `neuraloperator.FNO1d(width=4, n_layers=2, n_modes=32)` with the same hinge, compare MRR. Agreement within 2 % is the bar; it has not been run.

Part C — 32 Phical practice topics

```
        use phiano::phical::{Phical, ColorSpaceTimePoint,
ColorSpaceTimeManifold, PhicalOps, Topology};
        use phiano::phiton::{Phiton, Chiton, LightQuantum, SpectralBand};
        use phiano::phinum::{Hexagram, Trigram, SearleClassifier, Phinum64};
        use phiano::persona::{Persona, World, Fingerprint};
```

C1 · Phase to colour. `Phiton::phase_to_wavelength(1.284, 0)` → 455.6 nm; `LightQuantum::from_phase(1.284, 1.0, 0).to_color().hex()` → a blue. Every word in Phiano has a colour by this map; it is a coordinate choice, not physics.

C2 · Colour to phase. `Phiton::wavelength_to_phase(590.0)` → 3.566 rad. The inverse of C1 on [380, 750].

C3 · Frequency, period, energy. `Phiton::frequency_thz(455.6)` → 658 THz; `Phiton::energy_ev(455.6)` → 2.72 eV. These *are* physics (ν = c/λ, E = hc/λ).

C4 · Effective phase. `Phical::effective_phase(φ, n)` = φ + min(n, 13)/137 mod 2π. `n` on the lexicon is a convergence counter (1.8 M on the v1 manifold); `Phical::quantum(n)` is the sub-band number and every formula uses it.

C5 · A point on the manifold. `ColorSpaceTimePoint::from_phase(φ, n)` gives wavelength, frequency, effective phase, band. `.band()` and `.color()` name it.

C6 · Geodesic. `ColorSpaceTimeManifold::geodesic(&a, &b)` = √(Δθ² + φ_g(Δn·α)²) with Δθ wrapped. Two words at Δθ = π are as far apart as the torus allows.

C7 · Gradient and relax. `PhicalOps::relax_phase(θ_a, n_a, θ_b, n_b)` moves θ_a one `LEARNING_RATE` step along the unit tangent toward b. Iterate to interpolate; it is a Kuramoto step in manifold coordinates.

C8 · Curvature. `ColorSpaceTimeManifold::curvature(&p)` = α(n+1)/φ_g. Larger n, tighter curvature: the design's "more specific meaning". Check it means anything once n is fixed.

C9 · Standing-wave nodes. `ColorSpaceTimeManifold::is_standing_wave_node(&p)` — true when `bind_to_node` would move p by less than one channel quantum (2π/256). It used to return true for every p (it sampled cos(0)·cos(0)); the test `nodes_are_not_everywhere` guards the fix.

C10 · Bind to a node. `PhicalOps::bind_to_node(θ, n)` snaps θ to the nearest multiple of `GOLDEN_ANGLE·(1 + nα)`. Golden-angle nodes are the most evenly spread lattice on the circle that is not a regular polygon — the sunflower argument.

C11 · Interference. `PhicalOps::interference(θa, na, θb, nb)` = √(AaAb)·cos Δφ_eff. Positive = reinforce, negative = cancel. Use it to decide whether two personas sing together (Paper 3 ex. 14).

C12 · Spectral convolution. `PhicalOps::spectral_convolution(θa, na, θb, nb)`: wavelength overlap × chiton resonance × (1 + coupling/α). A similarity that includes "time" (frequency) as well as colour.

C13 · Temporal projection. `PhicalOps::temporal_project(&p)` → (ω, T). ω = 2πν in scaled units; T = 1/ν.

C14 · Chiton from phiton. `Chiton::from_quantum(&q)` flips the spin and derives ω, T from λ. `Chiton::resonance(&other)` is spin-matched, golden-weighted, α-damped. This is the "2-in-1" the module describes.

C15 · A sentence as a path. `Topology::evaluate_sentence(vertices)` → `Path { total_length, coherence, winding }`. Winding = Σ wrapped Δθ / 2π; a sentence that walks round the colour wheel has |winding| ≈ 1. On bare phases,

`phi_cal::winding::{winding, winding_closed, arc_length}` do the same without building vertices.

C16 · A story as a surface. `Topology::evaluate_story(paths)` → `Surface { area, genus }`. Genus is `max(0, 2 − (V − E − 1))` on the sentence-adjacency graph: a repetition heuristic with a topological name; label it so.

C17 · A concept as a region. `Topology::evaluate_concept(vertices)` → circular centroid, spread, density = 1/(1 + φ_g·spread). Feed a persona's top-20 words to get its concept density.

C18 · Relating two sentences without words. `Topology::layer_relation(&p, &q)` = coherence product × length ratio / (1 + α|Δwinding|). Zero for empty paths.

C19 · Sector to point. `ColorSpaceTimeManifold::sector_to_point(k, 64, n)` — phase 2πk/64 (sector *edge*; `phi_cal::q64::lift(k)` is the centred version and the one the juncture uses).

C20 · Hexagram of a phase. `Hexagram::from_phase(θ)` → 6-bit id, King Wen number, upper/lower trigrams. `.phase_angle()` is the lift with the half-step, identical to `phi_cal::q64::lift` (tested); the round trip errs by ≤ π/64. `phi_cal::q64::quantise_word(&phases)` gives a word's 64-symbol key and `hamming` compares two.

C21 · Spin and changing lines. `h.spin(Δθ)` moves along the circle; `h.changing_lines(0b000001)` flips one line. Symbolic (Hamming) distance vs geometric (geodesic) distance — measure how often they agree.

C22 · Speech act of a hexagram. `h.speech_act()` maps the upper trigram to a Searle act. Compare with `SearleClassifier::classify(sentence)` on real sentences: the agreement rate is the test of whether the mapping means anything.

C23 · Phinum64 classification. `Phinum64::classify_str("borrow")` → `n64#k`. A hash into 64 slots — a deterministic key, not a learned class. Use it for indexing, not for meaning.

C24 · Fingerprint a persona. `Fingerprint::extract(&facet, &examples)` → 64-bin histogram, dominant sectors, entropy, mean length. Rare words weigh `0.3/(1+amplitude)`.

C25 · Compare two personas. `world.compare("poet", "lawyer")` → similarity (cosine of histograms) and the top difference sectors with their colours. The differences are *what* they write about; syntax is not measured.

C26 · Attribute a text. `world.match_text(&facet, text)` → ranked likelihoods and a confidence `(best − second)/best`. Below 0.1 the system is guessing; say so in any UI.

C27 · Temperament. `fingerprint.personality_traits()` → warm/cool/green thresholds (0.4/0.4/0.3 of the top five). The thresholds are the arbitrary part.

C28 · Impersonate. `Impersonator::new().impersonate(&mut facet, &trainer, &fp, "poet", prompt)` — RiverFlows from dominant sectors, score 0.4·quality + 0.6·fit, ≤ 4 rounds. The lexicon is retrained on the best 8 flows each round; that is a

side effect to know about.

C29 · Persona colour and hexagram. Centroid of the histogram → `phase_to_wavelength` → colour; `Hexagram::from_phase(centroid)` → its Q64 name. Two one-liners that make a persona visible and nameable.

C30 · Stereographic lift. `use phiano::phical::exact::sphere; let p = sphere::lift(θ, n);` — `ρ = ln(1+n); p.latitude()` is evidence, `p.longitude()` is phase, `sphere::unstereo(p)` returns; `sphere::fibonacci_lattice(64)` and `nearest` give the cap alphabet. A persona's dominant sectors with their evidence become a cap. Overlap of caps is a persona distance that sees evidence; cosine of histograms does not.

C31 · Laplace–Beltrami smoothing. `fp_a.overlap_after(&fp_b, 0.2)` — both histograms diffused on the circle for time t (exact in Fourier modes, `phi_cal::heat`), then the Bhattacharyya overlap. At t = 0 disjoint personas overlap at 0; at t = 0.2 personas two sectors apart overlap above 0.5 (tested). The t that maximises attribution confidence on short texts is the persona's reach. `fp.q64()` is its 6-bit name.

C32 · Cross-channel coupling (proposed). Learn `K_kl` in `couple_relations`; accept only if offset+1 dispersion stays ≥ 0.85 and balanced similarity ≥ 51.8 % (Paper 1 §5.1). The same collapse that attraction caused is the risk.

What to run first

1. `cargo test -p phinet` — 10 tests + 1 doc test, all finite-difference.
2. `cargo run --release --bin phinet_residual` — the numbers in Paper 2 §3–4.
3. C9 and C4 are fixed; C10, C13, C15 mean what they say.
4. Parallel `cosine_table`; then WikiText-2 at the mixture objective, the experiment that decides whether the phase term ever matters.